H2 makes those tests less reliable. Each commit should be a candidate for release when adopting a continuous delivery approach. Suppose the autotests run by the deployment pipeline are not using the same backing services used in production. In that case, you'll need to do extra manual testing before deploying the application safely in production because you can't be sure it will work correctly. Therefore, it's essential to reduce the gap between environments.

Docker makes it easier to set up and develop applications with an actual database locally, as you experienced with PostgreSQL. In a similar way, Testcontainers (a Java library for testing) makes it easy to use backing services as containers in the context of integration tests.

This section will show you how to write slice tests for the data persistence layer using the `@DataJdbcTest` annotation and include a database in integration tests using the `@SpringBootTest` annotation. In both cases, you'll rely on Testcontainers to run the autotests against an actual PostgreSQL database.

5.3.1 Configuring Testcontainers for PostgreSQL

Testcontainers (https://testcontainers.org) is a Java library for testing. It supports JUnit and provides lightweight, throwaway containers such as databases, message brokers, and web servers. It's perfect for implementing integration tests with the actual backing services used in production. The result is more reliable and stable tests, which lead to higher-quality applications and favor continuous delivery practices.

You can configure a lightweight PostgreSQL container with Testcontainers and use it in your autotests involving a data persistence layer. Let's see how it works.